

Exercises 5.1–5.4 — Monte Carlo Methods

J.C. Vaught

Spring 2026 — CSCE 775

How Blackjack Works (Quick Primer)

If you have never played blackjack, here is all you need to know for this exercise.

The goal. Get a hand of cards whose values add up as close to 21 as possible, *without going over*. If your total exceeds 21 you “bust” and lose immediately.

Card values. Number cards (2–10) are worth their face value. Face cards (Jack, Queen, King) are each worth 10. An Ace is special: it can count as either 1 or 11, whichever helps you more.

A round of play. Both you (the player) and the dealer start with two cards. You can see one of the dealer’s cards (the “showing” card) but not the other. On your turn you have two choices:

- **Hit** — take another card (adds to your total, but risks going over 21).
- **Stick** (also called “stand”) — stop taking cards and keep your current total.

After you stick, the dealer plays by a fixed rule: keep hitting until reaching 17 or higher, then stick. Whoever is closer to 21 without busting wins.

Rewards. Win = +1, Lose = −1, Draw = 0. There is no discounting ($\gamma = 1$), so the reward from the end of the game *is* the return for every state visited during that game.

What is a “usable ace”? If you hold an Ace that you are currently counting as 11 (and your total is still ≤ 21), that Ace is called *usable*. It acts like a safety net: if a future card would push you over 21, you can “downgrade” the Ace from 11 to 1 and avoid busting. Once downgraded, the Ace is no longer usable.

The Policy Being Evaluated

The textbook evaluates a very simple (and not very good) policy:

If your hand totals 20 or 21, stick. Otherwise, hit.

This means the player keeps requesting cards until reaching 20 or 21 — risky, because every extra card could push the total over 21.

What the 3D plots show. Each surface plots the *value function* $V(s)$: how good, on average, is it to be in a particular situation? The axes are:

- **Player sum** (12–21) — your current card total.
- **Dealer showing** (Ace through 10) — the one dealer card you can see.
- **Height** (−1 to +1) — the average outcome from that situation. Positive means you tend to win; negative means you tend to lose.

There are two separate surfaces: one for hands *with* a usable ace, one *without*.

Problem Statement

Exercise 5.1 asks us to look at the 500,000-episode plots (the right-hand column of Figure 5.1) and explain three visual features:

1. Why does the surface jump *up* sharply for the last two rows in the rear (player sums 20 and 21)?
2. Why does the surface drop *down* along the entire left edge (dealer showing an Ace)?
3. Why are the front values (low player sums) *higher* in the usable-ace plot than in the no-usable-ace plot?

Reproduced Figure

The figure below was generated by simulating 10,000 and 500,000 blackjack games under the stick-on-20-or-21 policy and recording the average outcome for each state (first-visit Monte Carlo prediction).

Answer 1: The big jump at player sums 20 and 21

What is happening in the plot. The surface is mostly flat and negative (around -0.6 to -0.8) for player sums 12 through 19, then it suddenly shoots up to positive territory at sums 20 and 21.

Why. This jump happens because the policy *changes its behavior* right at sum 20. Think of it in two contrasting scenarios.

Player sum is 19 or below — the policy says “hit.” The player is forced to take another card. With a sum of 19, almost any card except an Ace or a 2 will push the total over 21 (a bust). Specifically, only 3 out of every 13 cards are safe; the other 10 cause an instant loss. So the average outcome from sum 19 is very bad (about -0.75).

Player sum is 20 — the policy says “stick.” Now the player stops and keeps their 20. To beat a 20, the dealer must end up with exactly 21. That is hard to do. So the player wins most of these games, and the average outcome jumps to roughly $+0.15$ to $+0.45$ depending on what the dealer is showing.

The table below shows this cliff from the simulation (no usable ace case):

Player Sum	Avg. outcome vs. Dealer Ace	Avg. outcome vs. Dealer 10
19	-0.779	-0.746
20	$+0.148$	$+0.436$
21	$+0.640$	$+0.888$

Table 1: The jump from sum 19 to sum 20 is nearly a full point — from almost-always-losing to usually-winning — because the policy switches from hit to stick.

In short: sums 20 and 21 are the only ones where the player actually *stops and keeps a strong hand*. Everything below 20 is dragged down by the constant risk of busting while hitting.

Exercise 5.1: Blackjack Value Function (stick on 20/21)

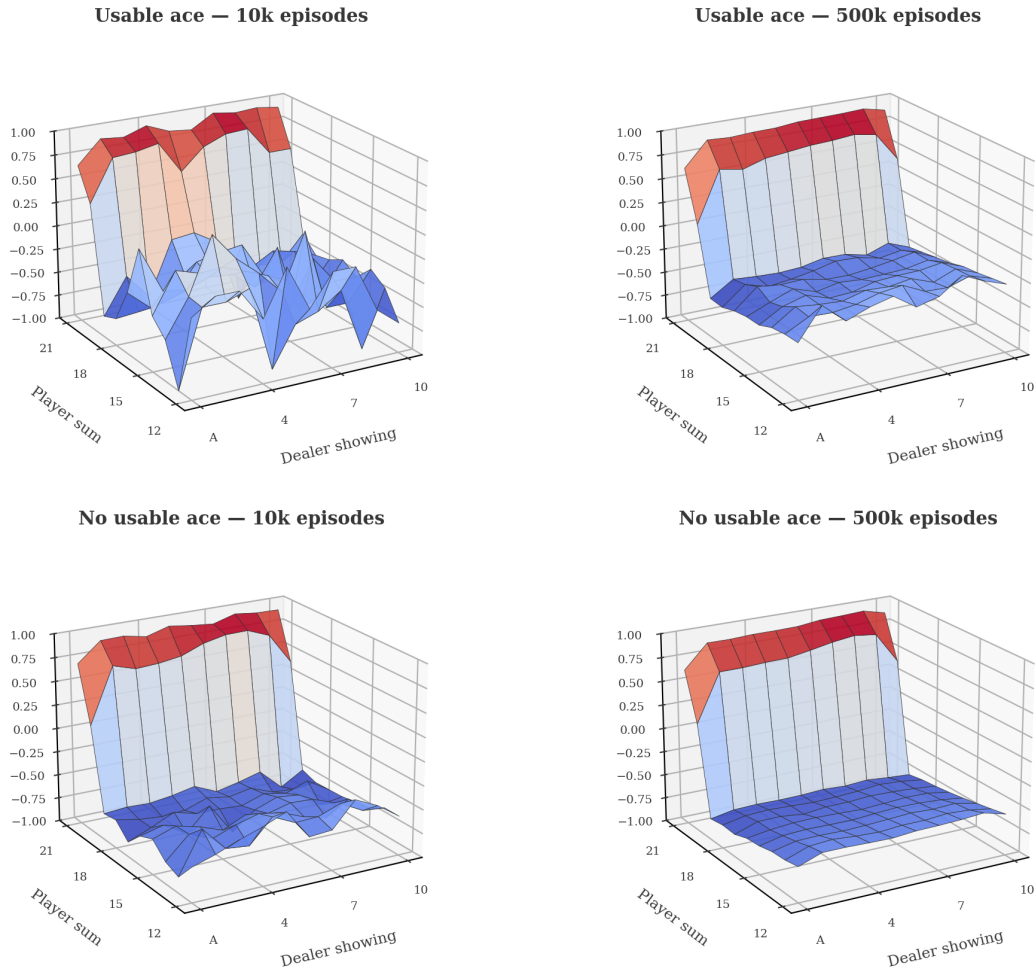


Figure 1: Approximate state-value functions for the stick-on-20-or-21 policy. The 10,000-episode surfaces (left) are noisy; the 500,000-episode surfaces (right) are smooth and accurate.

Answer 2: The drop when the dealer shows an Ace

What is happening in the plot. Along the left edge of each surface (where the dealer's showing card is an Ace), the values are noticeably lower than everywhere else.

Why. An Ace is the best possible card for the dealer, for an intuitive reason: *flexibility*.

Imagine the dealer flips over a 6 as their hidden card, giving them Ace + 6. They can count the Ace as 11 for a total of 17 and stick (a decent hand). Now imagine instead the hidden card is a 7, giving Ace + 7 = 18. Even better. And if a later hit would push the total over 21, the dealer just “downgrades” the Ace from 11 to 1 and keeps going — the Ace acts like a built-in safety net for the dealer too.

Because of this flexibility, the dealer almost never busts when starting with an Ace. The bust rate for a dealer showing an Ace is only about 11.5%. Compare that to a dealer showing a 5 or 6, where the bust rate is roughly 40%. When the dealer busts, the player wins automatically — so a low dealer bust rate means fewer free wins for the player.

In short: the dealer’s Ace makes the dealer’s hand very hard to beat, pulling the player’s average outcome down across *all* player sums.

Answer 3: Usable ace helps at low sums

What is happening in the plot. At the front of the surfaces (low player sums like 12, 13, 14), the *usable ace* plot sits noticeably higher than the *no usable ace* plot.

Why. Remember, under this policy the player hits on everything below 20. At low sums the player will have to hit *multiple times*, and each hit risks a bust. Having a usable ace is like wearing a seatbelt during this process.

Here is a concrete example. Suppose the player’s cards are Ace + 4, totaling 15 (counting the Ace as 11). The player hits and draws a 9. Normally $15 + 9 = 24$, which is a bust. But because the Ace is usable, it downgrades from 11 to 1, changing the total from 24 to 14. The player survives and gets to keep hitting.

Now consider the same situation *without* a usable ace: the player has, say, $8 + 7 = 15$. They hit and draw a 9. The total is $15 + 9 = 24$ — bust, instant loss, -1 reward. No safety net.

This difference shows up clearly in the numbers. At player sum 12 against a dealer showing 10:

$$V(\text{with usable ace}) = -0.279 \tag{1}$$

$$V(\text{without usable ace}) = -0.567 \tag{2}$$

With a usable ace the player still loses on average (negative value), but *much* less often, because the ace absorbs hits that would otherwise cause a bust.

As the player sum gets closer to 20, this advantage shrinks — there is less room for the ace to help, and at sums 20–21 the player sticks immediately regardless, so the usable ace barely matters.

In short: a usable ace is a “get out of bust free” card. It matters most at low sums where the player has to hit many times, making those states significantly less bad.

Summary

Feature	Plain-English Explanation
Jump at sums 20–21	These are the only sums where the player stops hitting and keeps a strong hand. Everything below 20 is terrible because the player is forced to keep drawing cards and often goes over 21.
Drop at dealer Ace	The Ace is the dealer’s best card because it is flexible (can count as 1 or 11). This means the dealer almost never busts, so the player gets fewer “free wins.”
Higher front values with usable ace	A usable ace lets the player survive hits that would otherwise cause a bust — it is a safety net. This matters most at low sums where the player has to hit many times before reaching 20.

Exercise 5.2

First-Visit vs. Every-Visit Monte Carlo

Problem Statement

Suppose every-visit MC was used instead of first-visit MC on the blackjack task. Would you expect the results to be very different? Why or why not?

Background: What is the difference?

To answer this, we first need to understand the two flavors of Monte Carlo prediction.

First-visit MC. When computing the value of a state s , we look at each simulated game (episode). If the player passed through state s during that game, we record the outcome — but only for the *first time* s appeared in that episode. If somehow the player landed in state s again later in the same game, we would ignore that second visit.

Every-visit MC. Same idea, except we count *every* time s appears in the episode, not just the first time. If s shows up three times in one game, we record the outcome three times and average all of them together.

When would this matter? The two methods only differ when the *same state is visited more than once within a single episode*. If every state only ever appears once per episode, the two methods are identical — there is nothing to “skip” under first-visit, because there are no repeat visits.

Answer: No, the results would not differ at all

In blackjack under this policy, **no state is ever visited twice in the same game**. The reason has to do with how the state is defined and how the game works.

Recall that a state is a triple: (player sum, dealer showing, usable ace). Consider what happens each time the player takes a card (hits):

- The **dealer’s showing card** never changes during a game — it is dealt once and stays face-up. So this part of the state is fixed.
- The player’s **usable ace** status can change, but only in one direction: a usable ace can be “spent” (downgraded from 11 to 1), but a spent ace cannot become usable again.
- The **player sum** changes with every hit. And here is the key: each card drawn has a value of at least 1, so the player sum *strictly increases* with every hit — unless the usable ace gets downgraded, in which case the sum drops by 10, but the usable-ace flag also flips. Either way, the overall state triple changes.

More concretely, think of it step by step. Suppose the player has sum 14 with no usable ace against a dealer showing 7 — state (14, 7, no). They hit and draw a 5, moving to sum 19 — state (19, 7, no). They hit again and draw a 3, moving to sum 22 — bust (game over). At no point did the player return to state (14, 7, no), because the sum can only go up.

The only tricky case is with a usable ace: suppose the player has sum 15 with a usable ace — state (15, 7, yes). They hit and draw a 9, pushing the total to 24. The ace downgrades (−10), bringing them to sum 14 — but now the state is (14, 7, no). The usable-ace flag is different, so this is a *different state* from before.

Since no state ever repeats within an episode, every visit *is* the first visit. The two methods produce **exactly identical results**.

Empirical Verification

To confirm this reasoning, we ran both first-visit and every-visit MC on the same 500,000 blackjack episodes. The difference between the two methods was verified to be exactly zero for all 200 states.

Some key statistics from the simulation:

- Total state visits recorded (first-visit): 777,567
- Total state visits recorded (every-visit): 777,567 (identical)
- Episodes with any state revisited: 0 out of 500,000 (0%)
- Maximum absolute difference across all states: 0.0000

When *would* they differ?

For completeness, the two methods *would* produce different results in environments where the agent can revisit the same state within an episode. For example:

- A robot navigating a gridworld that can walk in circles (visiting the same cell multiple times before reaching a goal).
- Any episodic task where the state can “reset” or cycle back to a previous value.

Even in those cases, both methods converge to the true value function given enough episodes — they just converge at slightly different rates. First-visit MC produces unbiased estimates. Every-visit MC is biased (slightly) but has lower variance, and the bias disappears as the number of episodes grows.

In blackjack, none of this matters because the two methods see exactly the same data.

Summary

Question	Answer
Would results differ?	No — the results are <i>exactly</i> identical.
Why?	Because no state is ever visited twice in a single blackjack episode. Each hit changes the player sum (always increases by at least 1) or flips the usable-ace flag, so the state triple always moves to a new value.
When would they differ?	In environments where the agent can revisit the same state within an episode (e.g., a gridworld with loops). Blackjack does not have this property.

Exercise 5.3

Backup Diagram for Monte Carlo Estimation of q_π

Problem Statement

What is the backup diagram for Monte Carlo estimation of q_π ?

Background: What is a backup diagram?

A *backup diagram* is a small picture that shows how an algorithm computes (or “backs up”) a value estimate. Think of it as a visual summary of the question: “What information does the algorithm use to update its estimate for one state (or state-action pair)?”

The diagrams use two shapes:

- **Open circle** $\circ$ — represents a *state*. This is a situation the agent finds itself in (e.g., “player sum is 15, dealer shows 7, no usable ace”).
- **Filled circle** $\bullet$ — represents a *state-action pair*. This is a situation *plus* a specific action (e.g., “player sum is 15, dealer shows 7, no usable ace, *and the player hits*”).
- **Filled square** — represents the *terminal state* (end of the episode, e.g., the game is over).

The node at the *top* of the diagram is the thing whose value we are estimating. Lines going downward show the information used to compute that estimate.

Three diagrams to compare

To understand the answer, it helps to see three related backup diagrams side by side.

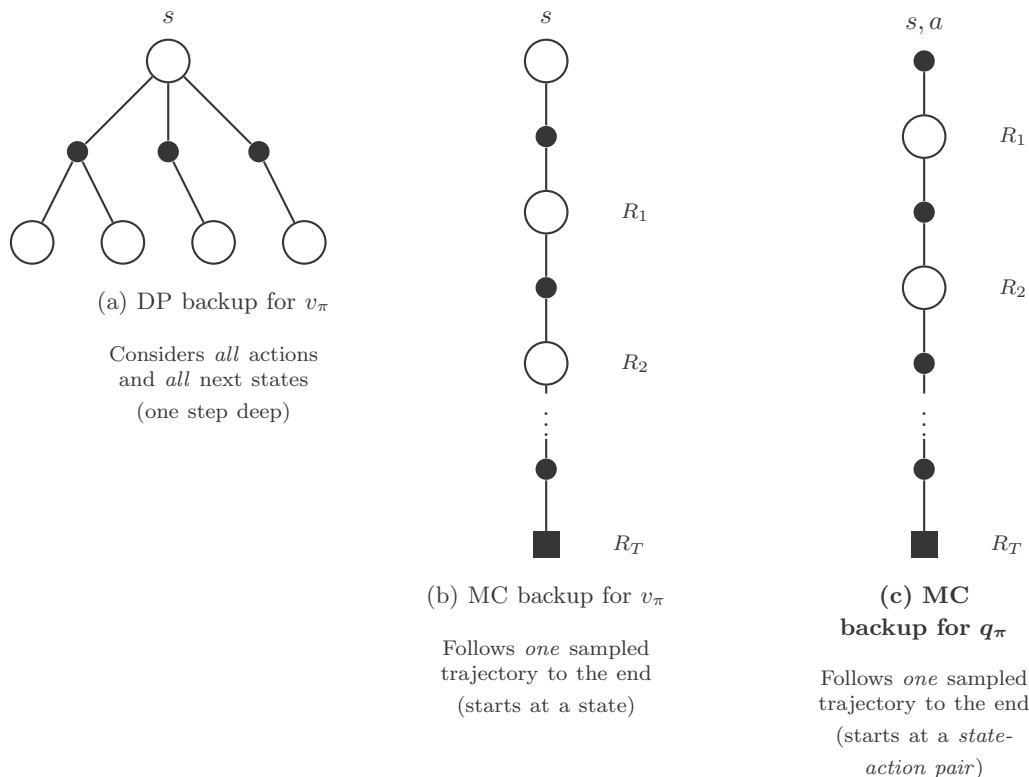


Figure 4: Three backup diagrams. (a) The DP diagram for v_π fans out over all actions and all next states but only goes one step deep. (b) The MC diagram for v_π follows a single sampled trajectory from a *state* all the way to the terminal state. (c) The MC diagram for q_π — the answer to Exercise 5.3 — is the same single trajectory, but starting from a *state-action pair* (filled circle) instead of a state (open circle).

Answer

The backup diagram for Monte Carlo estimation of q_π is diagram (c) above. It looks almost identical to the MC backup for v_π , with one key difference.

The difference: what is at the top.

- MC for v_π starts at an **open circle** (a state s). We are asking: “How good is it to *be in* this state?”
- MC for q_π starts at a **filled circle** (a state-action pair s, a). We are asking: “How good is it to *be in* this state *and take* this specific action?”

Everything below the top is the same. After the starting action is taken, the environment produces a reward and a next state, the policy picks the next action, and so on — a single sampled path all the way down to the terminal state (the filled square at the bottom). The return from that entire trajectory is used to update the estimate of $q_\pi(s, a)$.

Why does it look this way?

The shape of a backup diagram reflects two design choices:

1. **How wide is it?** (Does it branch, or is it a single path?)

- *DP diagrams are wide* — they fan out to consider *every* possible action and *every* possible next state. This is possible because DP has access to the full model (the transition probabilities $p(s', r \mid s, a)$).
- *MC diagrams are narrow* — they follow just *one* sampled trajectory. MC does not need a model; it just plays out one episode and sees what happens.

2. How deep is it? (Does it go one step or to the end?)

- *DP diagrams are shallow* — they look ahead exactly one step and then plug in an estimated value for the next state (bootstrapping).
- *MC diagrams are deep* — they go all the way to the end of the episode. MC does *not* bootstrap; it waits for the actual outcome.

For q_π specifically, the diagram starts one level lower than v_π , because we have already committed to a particular action. The first transition out of the top node is the environment's response to that action (producing a reward and a next state), not the policy choosing which action to take.

Summary

Diagram	Top node	Shape	What it does
DP for v_π	Open circle (s)	Wide, shallow	Branches over all actions and next states, one step deep
MC for v_π	Open circle (s)	Narrow, deep	Single trajectory from state to terminal
MC for q_π	Filled circle (s, a)	Narrow, deep	Single trajectory from state-action pair to terminal

Exercise 5.4

Incremental Monte Carlo ES

Problem Statement

The pseudocode for Monte Carlo ES is inefficient because, for each state-action pair, it maintains a list of all returns and repeatedly calculates their mean. It would be more efficient to use techniques similar to those explained in Section 2.4 to maintain just the mean and a count (for each state-action pair) and update them incrementally. Describe how the pseudocode would be altered to achieve this.

Background: Why is the original version inefficient?

The Monte Carlo ES algorithm on page 99 of the textbook stores data like this:

$Returns(s, a) \leftarrow$ an ever-growing list of numbers

Every time the agent finishes a game and visits state-action pair (s, a) , the algorithm appends the return G to the list $Returns(s, a)$ and then recomputes the average of the *entire* list. As the agent plays more and more games, these lists get longer and longer. After 500,000 games, a frequently visited pair might have a list with tens of thousands of entries, and the algorithm recomputes the average over all of them every single time.

This is wasteful in two ways:

- **Memory:** We are storing every return we have ever seen. For 200 state-action pairs visited thousands of times each, that is a lot of numbers sitting in memory.
- **Computation:** Recomputing the average of n numbers takes $O(n)$ time. As n grows, this gets slower and slower. But the new average only differs from the old average by a tiny adjustment — surely we can compute it faster.

The trick from Section 2.4: Incremental mean update

Section 2.4 of the textbook shows that you do not need to store the entire list. You only need two things: the *current average* and a *count* of how many values you have seen so far.

The key formula (Equation 2.3 in the textbook) is:

$$Q_{n+1} = Q_n + \frac{1}{n} [R_n - Q_n] \quad (3)$$

In plain English: to update the average after seeing a new value, take the old average Q_n , compute the “error” (how far the new value R_n is from the old average), and nudge the average toward the new value by a fraction $1/n$.

This is an instance of the general update pattern that appears throughout reinforcement learning:

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \text{StepSize} \times [\text{Target} - \text{OldEstimate}] \quad (4)$$

The step size $1/n$ gets smaller as we see more data, which makes sense: early observations should move the average a lot, but after thousands of observations, each new one should barely budge it.

Answer: The modified pseudocode

We replace the $Returns(s, a)$ lists with two simpler data structures: $Q(s, a)$ (the running average) and $N(s, a)$ (the count). Here is the full modified algorithm:

Monte Carlo ES (Incremental), for estimating $\pi \approx \pi_*$

Initialize:

- $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
- $Q(s, a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
- $N(s, a) \leftarrow 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$ $\triangleleft$ replaces *Returns*(s, a) lists

Loop forever (for each episode):

- Choose $S_0 \in \mathcal{S}, A_0 \in \mathcal{A}(S_0)$ randomly (exploring starts)
- Generate an episode from S_0, A_0 , following π :
 $S_0, A_0, R_1, S_1, A_1, R_2, \dots, S_{T-1}, A_{T-1}, R_T$
- $G \leftarrow 0$
- Loop** for each step of episode, $t = T-1, T-2, \dots, 0$:
 - $G \leftarrow \gamma G + R_{t+1}$
 - Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1, \dots, S_{t-1}, A_{t-1}$:
 - $N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$ $\triangleleft$ increment count
 - $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)} [G - Q(S_t, A_t)]$ $\triangleleft$ incremental update
 - $\pi(S_t) \leftarrow \arg \max_a Q(S_t, a)$

What changed, step by step

Three things changed from the original pseudocode:

1. Initialization.

- *Removed:* $Returns(s, a) \leftarrow$ empty list, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
- *Added:* $N(s, a) \leftarrow 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}(s)$
- $Q(s, a)$ was already initialized (arbitrarily). No change needed there.

2. Inside the loop (when a first-visit state-action pair is found).

- *Removed:* “Append G to $Returns(S_t, A_t)$ ”
- *Removed:* “ $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$ ”
- *Added:* “ $N(S_t, A_t) \leftarrow N(S_t, A_t) + 1$ ”
- *Added:* “ $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)} [G - Q(S_t, A_t)]$ ”

3. Everything else stays the same. The episode generation, the return calculation ($G \leftarrow \gamma G + R_{t+1}$), the first-visit check, and the policy improvement step ($\pi(S_t) \leftarrow \arg \max_a Q(S_t, a)$) are all unchanged.

Why this works

The incremental formula produces *exactly* the same running average as recomputing from the full list every time. Here is the intuition with a small example.

Suppose we have seen three returns for a particular state-action pair: 5, 3, and 7.

List-based approach:

$$Q = \text{average}(\{5, 3, 7\}) = \frac{5 + 3 + 7}{3} = 5.0 \quad (5)$$

Incremental approach:

$$N = 1, \quad Q = 0 + \frac{1}{1}[5 - 0] = 5.0 \quad (6)$$

$$N = 2, \quad Q = 5.0 + \frac{1}{2}[3 - 5.0] = 5.0 + (-1.0) = 4.0 \quad (7)$$

$$N = 3, \quad Q = 4.0 + \frac{1}{3}[7 - 4.0] = 4.0 + 1.0 = 5.0 \quad (8)$$

Same result, but we never stored the list $\{5, 3, 7\}$. We only kept track of Q and N .

Complexity comparison

	Original (list-based)	Incremental
Memory per (s, a) pair	$O(n)$ — grows forever	$O(1)$ — just Q and N
Time per update	$O(n)$ — re-averages list	$O(1)$ — one multiply, one add
Total memory	Unbounded	$2 \mathcal{S} \mathcal{A} $ (fixed)

Table 2: The incremental version uses constant memory and constant time per update, regardless of how many episodes have been run.

Summary

Replace the ever-growing $Returns(s, a)$ lists with a simple counter $N(s, a)$. Instead of appending G to a list and recomputing the average, increment the counter and nudge the current estimate $Q(s, a)$ toward G using the step size $1/N(s, a)$. This produces identical results with $O(1)$ memory and $O(1)$ computation per update instead of $O(n)$ for both.

Exercise 5.2: First-Visit vs Every-Visit MC (500k episodes)

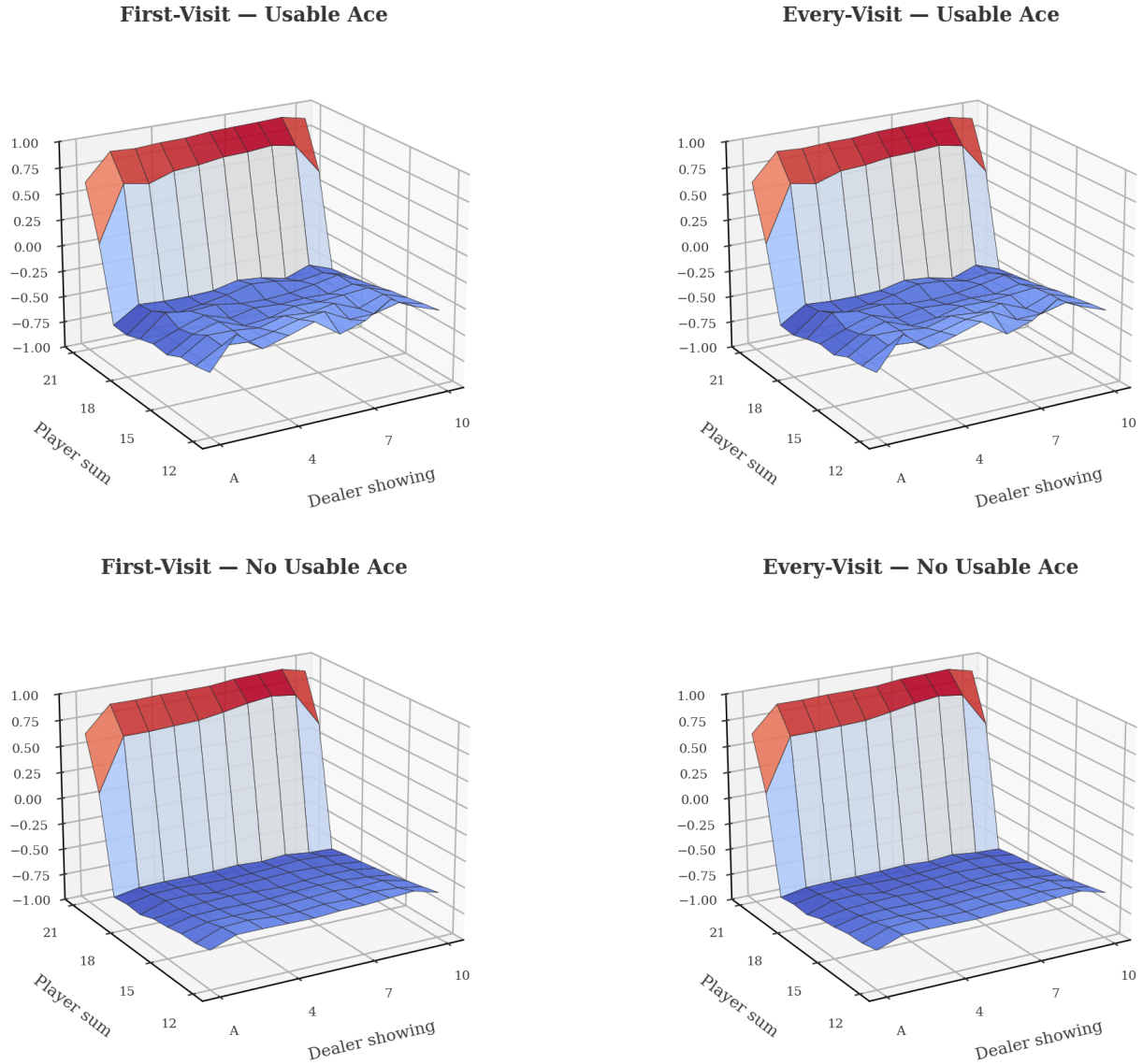


Figure 2: Side-by-side comparison of first-visit (left) and every-visit (right) MC surfaces after 500,000 episodes. The surfaces are visually identical because no state is ever revisited within a blackjack episode.

Absolute Difference: First-Visit minus Every-Visit MC

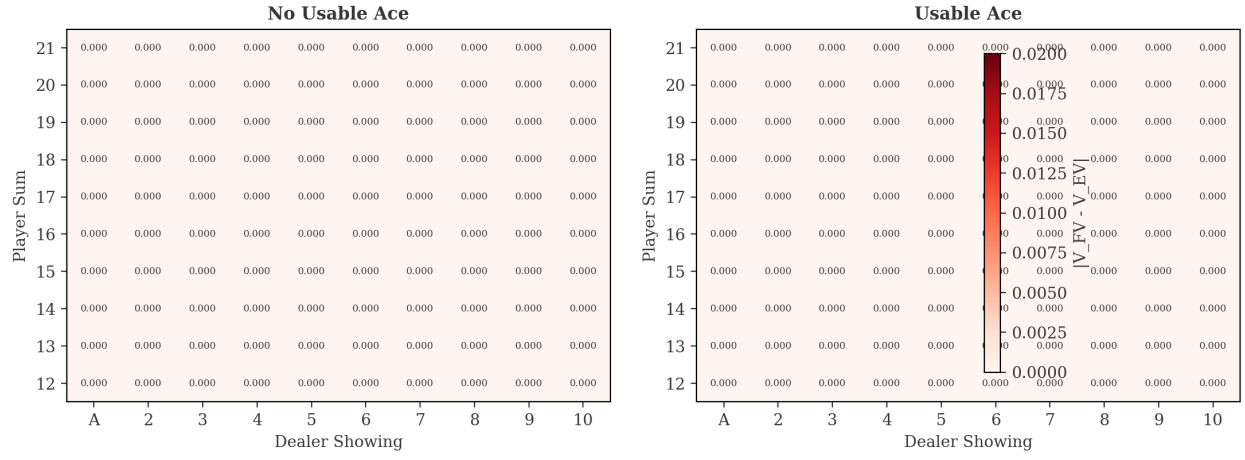


Figure 3: Absolute difference $|V_{\text{first-visit}} - V_{\text{every-visit}}|$ for each state. Every cell is exactly 0.000 — the methods produced identical estimates for all 200 states.